```c
#ifndef ALGS_H
#define ALGS_H

#include "../Groups/group.h"
#include "../Gen_Table/gen_table.h"
#include "../Garbage_Collector/garbage_collector.h"
#include "../Linked_List/linked_list.h"
#include "snf.h"
#include "algs.h"
#include <stdbool.h>

int find_neutral(group g);
int fast_exp(group g, int x, int neutral, int n);

int calculate_next_gen_and_rel(group g, group_data gd, int new_gen, int neutral);
int calculate_next_gen_and_rel_fast(group g, group_data gd, gbg_collector coll, int new_gen,
int neutral);

group_data calculate_group_data(group g);
group_data calculate_group_data_fast(group g, gbg_collector trash);

void check_group_data(group g, group_data gd, int neutral);

void print_arr(int* arr, int n);

matrix get_trig_rel_table(group g, int neutral, group_data gd);

typedef struct {
 int n;
 int* order;
 int* elts;
} basis_s;
typedef basis_s* basis;

basis create_empty_basis(int n);
void free_basis(basis b);
void check_basis(group g, basis b, int neutral);
void show_basis(basis b);

basis get_basis_from_data(group g, group_data gd, int neutral, matrix D, matrix V_inv);
basis get_basis_from_rel(group g, group_data gd, int neutral);
basis get_basis(group g);

void iterate_decomps(int* decomps, int dim, int i, int* orders);
bool are_isomorphic(group g1, group g2);
int* calculate_isomorphism(group g1, group g2);

bool check_isomorphism(group g1, group g2, int* iso);
#endif
```

```c
#include "algs.h"

int find_neutral(group g) {
 int next = 0;
 for (int i = 0; i < g->n; i++) {
  next = (i+1) % g->n;
  if (calculate(g, i, next) == next) {
   return i;
  }
 }
 printf("[-] Erreur.\n");
 return -1;
}

int fast_exp(group g, int x, int neutral, int n) {
 assert(g != NULL && x >= 0 && x < g->n);
 assert(is_neutral(g, neutral));
 if (n >= 0) {
  int res = neutral;
  int a = x;
  int p = (n % g->n);
  while (p > 0) {
   if (p % 2 == 1) {
    res = calculate(g, res, a);
   }
   a = calculate(g, a, a);
   p /= 2;
  }

  return res;
 } else {
  int p = (n % g->n) + g->n;
  return fast_exp(g, x, neutral, p);
 }
}

// Calcule les relations étant donné un nouveau générateur.
int calculate_next_gen_and_rel(group g, group_data gd, int new_gen, int neutral) {
 assert(g != NULL && gd != NULL);
 assert(new_gen >= 0);

 if (!gd->initialised) {
  init_empty_group_data(gd, neutral);
 }
 list new_grp_elts = create_empty_list();
 int curr = new_gen;
 int k = 1;
 int to_add, x;

 while (!gd->contains[curr]) {
  for (node_t* node = gd->grp_elts->head; node != NULL; node = node->next) {
   x = node->val;
   to_add = calculate(g, curr, x);

   if (!gd->contains[curr]) {
    push(new_grp_elts, curr);
    push_data_list(gd->decomps[curr], new_gen, k);
    gd->contains[curr] = true;
   }

   if (!gd->contains[to_add]) {
    push(new_grp_elts, to_add);
    del_copy_from(&gd->decomps[to_add], gd->decomps[x]);
    push_data_list(gd->decomps[to_add], new_gen, k);
    gd->contains[to_add] = true;
   }
  }
```

```
  k++;
  curr = calculate(g, curr, new_gen);
 }

 push_data_list(gd->generators, new_gen, k);

 int to_push;
 while(new_grp_elts->length > 0) {
  to_push = pop(new_grp_elts);
  push(gd->grp_elts, to_push);
 }
 free_list(new_grp_elts);

 int next_gen = 0;
 while (next_gen < gd->n && gd->contains[next_gen]) {
  next_gen++;
 }

 return next_gen;
}

int calculate_next_gen_and_rel_fast(group g, group_data gd, gbg_collector coll, int new_gen,
int neutral) {
 assert(g != NULL && gd != NULL);
 assert(new_gen >= 0);

 if (!gd->initialised) {
  init_empty_group_data(gd, neutral);
 }
 list new_grp_elts = create_empty_list();
 int curr = new_gen;
 int k = 1;
 int to_add, x;

 while (!gd->contains[curr]) {
  for (node_t* node = gd->grp_elts->head; node != NULL; node = node->next) {
   x = node->val;
   to_add = calculate(g, curr, x);

   if (!gd->contains[curr]) {
    push(new_grp_elts, curr);
    // push_data_list(gd->decomps[curr], new_gen, k);
    append_data_in_tabl_only(gd->decomps, gd->n, curr, new_gen, k, coll);
    gd->contains[curr] = true;
   }

   if (!gd->contains[to_add]) {
    push(new_grp_elts, to_add);
    // del_copy_from(&gd->decomps[to_add], gd->decomps[x]);
    append_data_and_link_in_tabl(gd->decomps, gd->n, to_add, x, new_gen, k, coll);
    gd->contains[to_add] = true;
   }
  }

  k++;
  curr = calculate(g, curr, new_gen);
 }

 push_data_list(gd->generators, new_gen, k);

 int to_push;
 while(new_grp_elts->length > 0) {
  to_push = pop(new_grp_elts);
  push(gd->grp_elts, to_push);
 }
 free_list(new_grp_elts);
```

```
  int next_gen = 0;
  while (next_gen < gd->n && gd->contains[next_gen]) {
   next_gen++;
  }

  return next_gen;
}


group_data calculate_group_data(group g) {
  int neutral = find_neutral(g);

  group_data gd = create_empty_group_data(g->n);
  init_empty_group_data(gd, neutral);

  int gen = 0;

  while (gen < g->n && gd->grp_elts->length < g->n) {
   gen = calculate_next_gen_and_rel(g, gd, gen, neutral);
  }

  return gd;
}

group_data calculate_group_data_fast(group g, gbg_collector trash) {
  int neutral = find_neutral(g);

  group_data gd = create_empty_group_data(g->n);
  init_empty_group_data(gd, neutral);

  int gen = 0;

  while (gen < g->n && gd->grp_elts->length < g->n) {
   gen = calculate_next_gen_and_rel_fast(g, gd, trash, gen, neutral);
  }

  return gd;
}

void check_group_data(group g, group_data gd, int neutral) {
  for (int i = 0; i < g->n; i++) {
   int res = neutral;
   for (node_data c = gd->decomps[i]->head; c != NULL; c=c->next) {
    int val = fast_exp(g, c->g, neutral, c->p);
    res = calculate(g, res, val);
   }
   if (i != res) {
    printf("[-] ERROR: %d != %d.\n", i, res);
   }
   assert(res == i);
  }
}

void print_arr(int* arr, int n) {
  for (int i = 0; i < n; i++) {
   printf("%d ", arr[i]);
  }
  printf("\n");
}


matrix get_trig_rel_table(group g, int neutral, group_data gd) {
  assert(gd != NULL && gd->generators != NULL);

  int t = gd->generators->length;
  matrix trig_rel = mat_alloc(t);
```

```c
 int *col_of = malloc((size_t)g->n * sizeof(int));
 assert(col_of != NULL);
 for (int x = 0; x < g->n; x++) col_of[x] = -1;

 int col = 0;
 for (node_data gen = gd->generators->head; gen != NULL; gen = gen->next) {
  assert(gen->g >= 0 && gen->g < g->n);
  col_of[gen->g] = col++;
 }
 assert(col == t);

 int i = 0;
 for (node_data gen = gd->generators->head; gen != NULL; gen = gen->next) {
  int curr_elt = fast_exp(g, gen->g, neutral, gen->p);
  data_list decomp = gd->decomps[curr_elt];

  trig_rel->t[i][i] -= gen->p;

  for (node_data c = decomp->head; c != NULL; c = c->next) {
   assert(c->g >= 0 && c->g < g->n);
   int j = col_of[c->g];
   assert(j >= 0 && j < t);
   trig_rel->t[i][j] += c->p;
  }
  i++;
 }

 free(col_of);
 return trig_rel;
}

basis create_empty_basis(int n) {
 basis b = malloc(sizeof(basis_s));
 b->n = n;
 b->elts = calloc(n, sizeof(int));
 b->order = calloc(n, sizeof(int));
 return b;
}
void free_basis(basis b) {
 free(b->elts);
 free(b->order);
 free(b);
}

void check_basis(group g, basis b, int neutral) {
 assert(is_neutral(g, neutral));
 for (int i = 0; i < b->n; i++) {
  int val = fast_exp(g, b->elts[i], neutral, b->order[i]);
  assert(val == neutral);
 }
}

void show_basis(basis b) {
 printf("Order:\n");
 print_arr(b->order, b->n);
 printf("Elements:\n");
 print_arr(b->elts, b->n);
}

basis get_basis_from_data(group g, group_data gd, int neutral, matrix D, matrix V_inv) {
 int t = gd->generators->length;
 printf("Generators: ");
 show_values_data_list(gd->generators);
 printf("\n");
 basis b = create_empty_basis(t);
 for (int i = 0; i < t; i++) {
```

```c
   b->elts[i] = neutral;
   int j = 0;
   for (node_data gen = gd->generators->head; gen != NULL; gen=gen->next) {
    int to_add = fast_exp(g, gen->g, neutral, ((V_inv->t[i][j]) % g->n) + g->n);
    b->elts[i] = calculate(g, b->elts[i], to_add);
    j++;
   }
   b->order[i] = D->t[i][i];
  }

 return b;
}

basis get_basis_from_rel(group g, group_data gd, int neutral) {
 int t = gd->generators->length;
 matrix R = get_trig_rel_table(g, neutral, gd);
 matrix U = identiy(t);
 matrix V = identiy(t);
 matrix D = mat_alloc(t);
 mat_snf_with_diagonal(R, U, V, D);
 // mat_print("U", U);
 // mat_print("V", V);
 // mat_print("D", D);

 matrix V_inv = mat_inverse(V);
 // mat_print("V inverse", V_inv);
 mat_free(R);
 mat_free(U);
 mat_free(V);

 basis b = get_basis_from_data(g, gd, neutral, D, V_inv);
 mat_free(D);
 mat_free(V_inv);

 return b;
}

basis get_basis(group g) {
 int neutral = find_neutral(g);
 printf("[*] En train de calculer les données du groupe.\n");
 group_data gd = calculate_group_data(g);
 basis b = get_basis_from_rel(g, gd, neutral);
 free_group_data(gd);
 printf("Base: ");
 print_arr(b->elts, b->n);

 printf("Ordres: ");
 print_arr(b->order, b->n);

 return b;
}

bool are_isomorphic(group g1, group g2) {
 int neutral1 = find_neutral(g1);
 group_data gd1 = calculate_group_data(g1);
 matrix R1 = get_trig_rel_table(g1, neutral1, gd1);
 free_group_data(gd1);

 matrix U1 = identity(R1->n);
 matrix V1 = identity(R1->n);
 matrix D1 = mat_alloc(R1->n);

 mat_snf_with_diagonal(R1, U1, V1, D1);

 mat_free(R1);
 mat_free(U1);
 mat_free(V1);
```

```c
 int neutral2 = find_neutral(g2);
 group_data gd2 = calculate_group_data(g2);
 matrix R2 = get_trig_rel_table(g2, neutral2, gd2);
 free_group_data(gd2);

 matrix U2 = identity(R2->n);
 matrix V2 = identity(R2->n);
 matrix D2 = mat_alloc(R2->n);

 mat_snf_with_diagonal(R2, U2, V2, D2);

 mat_free(R2);
 mat_free(U2);
 mat_free(V2);

 bool res = same_abelian_group_from_snf(D1, D2);

 mat_free(D1);
 mat_free(D2);

 return res;
}

void iterate_decomps(int* decomps, int dim, int i, int* orders) {
 if (i >= dim) {
  return;
 } else {
  if ((decomps[i] + 1) % orders[i] == 0) {
   decomps[i] = 0;
   iterate_decomps(decomps, dim, i+1, orders);
  } else {
   decomps[i] += 1;
  }
 }
}


int* calculate_isomorphism(group g1, group g2) {
 int neutral1 = find_neutral(g1);
 group_data gd1 = calculate_group_data(g1);
 basis b1 = get_basis_from_rel(g1, gd1, neutral1);

 int neutral2 = find_neutral(g2);
 group_data gd2 = calculate_group_data(g2);
 basis b2 = get_basis_from_rel(g2, gd2, neutral2);

 int* iso = malloc(g1->n*sizeof(int));
 for (int i = 0; i < g1->n; i++) {
  iso[i] = -1;
 }
 int i = b1->n - 1;
 int j = b2->n - 1;
 int dim = 0;

 while (b1->elts[i] != neutral1 && i >= 0 && b2->elts[j] != neutral2 && j >= 0) {
  iso[b1->elts[i]] = b2->elts[j];
  i--;
  j--;
  dim++;
 }

 int* real_basis_g1 = calloc(dim, sizeof(int));
 int* real_basis_g2 = calloc(dim, sizeof(int));
 int* orders = calloc(dim, sizeof(int));

 for (int i = 0; i < dim; i++) {
```

```
   real_basis_g1[i] = b1->elts[b1->n-1 - i];
   real_basis_g2[i] = b2->elts[b2->n-1 - i];
   orders[i] = b1->order[b1->n-1 - i];
  }

  int* decomps = calloc(dim, sizeof(int));

  for (int k = 0; k < g1->n; k++) {
   int curr_elt_g1 = neutral1;
   int curr_elt_g2 = neutral2;
   for (int i = 0; i < dim; i++) {
    int vec1 = real_basis_g1[i];
    int vec2 = real_basis_g2[i];
    curr_elt_g1 = calculate(g1, curr_elt_g1, fast_exp(g1, vec1, neutral1, decomps[i]));
    curr_elt_g2 = calculate(g2, curr_elt_g2, fast_exp(g2, vec2, neutral2, decomps[i]));
   }
   iso[curr_elt_g1] = curr_elt_g2;

   iterate_decomps(decomps, dim, 0, orders);
  }

  free(decomps);
  free(real_basis_g1);
  free(real_basis_g2);
  free(orders);

  free_group_data(gd1);
  free_group_data(gd2);

  free_basis(b1);
  free_basis(b2);

  return iso;
 }

 bool check_isomorphism(group g1, group g2, int* iso) {
  assert(g1->n == g2->n);
  int n = g1->n;
  for (int x = 0; x < n; x++) {
   for (int y = 0; y < n; y++) {
    int e_g1 = iso[calculate(g1, x, y)];
    int e_g2 = calculate(g2, iso[x], iso[y]);
    bool check = (e_g1 == e_g2);
    if (!check) {
     printf("[-] For (%d, %d): %d != %d.\n", x, y, e_g1, e_g2);
     return false;
    }
   }
  }

  return true;
 }
```

```c
#ifndef SMITH_NORMAL_FORM_H
#define SMITH_NORMAL_FORM_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

typedef long long ll;

typedef struct {
 int  n;
 ll **t;
} matrix_s;

typedef matrix_s *matrix;

matrix mat_alloc(int n);
void set_matrix(int n, matrix M, int m[][n]);
matrix from_given_matrix(int n, int m[][n]);
matrix identiy(int n);
matrix identity(int n);
void mat_free(matrix A);

void mat_identity(matrix A);
void mat_print(const char *name, matrix A);

void smith_normal_form(matrix M, matrix U, matrix V);

void mat_mul(matrix C, matrix A, matrix B);
matrix get_prod(matrix A, matrix B);
bool mat_equal(matrix A, matrix B);
bool same_abelian_group_from_snf(matrix D1, matrix D2);

int mat_inverse_bareiss(matrix M, matrix adj, ll *det_out);
matrix mat_inverse(matrix M);
matrix mat_snf_V_inv(matrix R);
void mat_snf_with_diagonal(matrix M, matrix U, matrix V, matrix D);
#endif
```

```c
#include "snf.h"

static ll extended_gcd(ll a, ll b, ll *x, ll *y) {
 if (b == 0) {
  *x = 1;
  *y = 0;
  return a;
 }
 ll x1, y1, g;

 g  = extended_gcd(b, a % b, &x1, &y1);
 *x = y1;
 *y = x1 - (a / b) * y1;
 return g;
}

matrix mat_alloc(int n) {
 if (n <= 0)
  return NULL;

 matrix A = (matrix)malloc(sizeof(matrix_s));
 if (A == NULL)
  return NULL;

 A->n = n;
 A->t = (ll **)calloc((size_t)n, sizeof(ll *));
 if (A->t == NULL) {
  free(A);
  return NULL;
 }

 int i = 0;
 while (i < n) {
  A->t[i] = (ll *)calloc((size_t)n, sizeof(ll));
  if (A->t[i] == NULL) {
   while (--i >= 0)
    free(A->t[i]);
   free(A->t);
   free(A);
   return NULL;
  }
  i++;
 }
 return A;
}

void set_matrix(int n, matrix M, int m[][n]) {
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   M->t[i][j] = m[i][j];
  }
 }
}

matrix from_given_matrix(int n, int m[][n]) {
 matrix M = mat_alloc(n);
 set_matrix(n, M, m);
 return M;
}

matrix identiy(int n) {
 matrix res = mat_alloc(n);
 for(int i = 0; i < n; i++) {
  res->t[i][i] = 1;
 }
 return res;
}
```

```c
matrix identity(int n) {
 return identiy(n);
}

void mat_free(matrix A) {
 if (A == NULL)
  return;
 int i = 0;
 while (i < A->n) {
  free(A->t[i]);
  i++;
 }
 free(A->t);
 free(A);
}

void mat_identity(matrix A) {
 int i = 0;
 while (i < A->n) {
  int j = 0;
  while (j < A->n) {
   A->t[i][j] = (i == j) ? 1 : 0;
   j++;
  }
  i++;
 }
}

void mat_print(const char *name, matrix A) {
 int i, j;

 printf("%s (%dx%d):\n", name, A->n, A->n);
 i = 0;
 while (i < A->n) {
  printf("  [");
  j = 0;
  while (j < A->n) {
   printf(" %4lld", A->t[i][j]);
   j++;
  }
  printf(" ]\n");
  i++;
 }
}

static void swap_rows(matrix A, matrix U, int i, int j) {
 int k;

 if (i == j)
  return;
 k = 0;
 while (k < A->n) {
  ll t       = A->t[i][k];
  A->t[i][k] = A->t[j][k];
  A->t[j][k] = t;
  k++;
 }
 if (U) {
  k = 0;
  while (k < U->n) {
   ll t       = U->t[i][k];
   U->t[i][k] = U->t[j][k];
   U->t[j][k] = t;
   k++;
  }
 }
```

```c
}

static void swap_cols(matrix A, matrix V, int i, int j) {
 int k;

 if (i == j)
  return;
 k = 0;
 while (k < A->n)
 {
  ll t        = A->t[k][i];
  A->t[k][i]  = A->t[k][j];
  A->t[k][j]  = t;
  k++;
 }
 if (V)
 {
  k = 0;
  while (k < V->n)
  {
   ll t        = V->t[k][i];
   V->t[k][i]  = V->t[k][j];
   V->t[k][j]  = t;
   k++;
  }
 }
}

static void row_combine(matrix A, matrix U, int r0, int r1, ll a, ll b, ll c, ll d) {
 int k = 0;
 while (k < A->n) {
  ll v0 = A->t[r0][k];
  ll v1 = A->t[r1][k];

  A->t[r0][k] = a * v0 + b * v1;
  A->t[r1][k] = c * v0 + d * v1;
  k++;
 }
 if (U) {
  k = 0;
  while (k < U->n) {
   ll v0 = U->t[r0][k];
   ll v1 = U->t[r1][k];

   U->t[r0][k] = a * v0 + b * v1;
   U->t[r1][k] = c * v0 + d * v1;
   k++;
  }
 }
}

static ll ll_abs_snf(ll x) {
 return (x < 0) ? -x : x;
}

static void add_row_multiple(matrix A, matrix U, int dst, int src, ll k) {
 if (k == 0 || dst == src)
  return;
 for (int j = 0; j < A->n; j++)
  A->t[dst][j] += k * A->t[src][j];
 if (U) {
  for (int j = 0; j < U->n; j++)
   U->t[dst][j] += k * U->t[src][j];
 }
}

static void add_col_multiple(matrix A, matrix V, int dst, int src, ll k) {
```

```c
 if (k == 0 || dst == src)
  return;
 for (int i = 0; i < A->n; i++)
  A->t[i][dst] += k * A->t[i][src];
 if (V) {
  for (int i = 0; i < V->n; i++)
   V->t[i][dst] += k * V->t[i][src];
 }
}

static void negate_row(matrix A, matrix U, int row) {
 for (int j = 0; j < A->n; j++)
  A->t[row][j] = -A->t[row][j];
 if (U) {
  for (int j = 0; j < U->n; j++)
   U->t[row][j] = -U->t[row][j];
 }
}

static int has_offdiag_in_pivot_cross(matrix M, int p) {
 for (int i = p + 1; i < M->n; i++)
  if (M->t[i][p] != 0 || M->t[p][i] != 0)
   return 1;
 return 0;
}

static void reduce_pivot_cross(matrix M, matrix U, matrix V, int p) {
 int n = M->n;

 while (has_offdiag_in_pivot_cross(M, p)) {
  for (int i = p + 1; i < n; i++) {
   while (M->t[i][p] != 0) {
    if (ll_abs_snf(M->t[i][p]) < ll_abs_snf(M->t[p][p]))
     swap_rows(M, U, p, i);
    ll q = M->t[i][p] / M->t[p][p];
    if (q == 0)
     q = (M->t[i][p] > 0) == (M->t[p][p] > 0) ? 1 : -1;
    add_row_multiple(M, U, i, p, -q);
   }
  }

  for (int j = p + 1; j < n; j++) {
   while (M->t[p][j] != 0) {
    if (ll_abs_snf(M->t[p][j]) < ll_abs_snf(M->t[p][p]))
     swap_cols(M, V, p, j);
    ll q = M->t[p][j] / M->t[p][p];
    if (q == 0)
     q = (M->t[p][j] > 0) == (M->t[p][p] > 0) ? 1 : -1;
    add_col_multiple(M, V, j, p, -q);
   }
  }
 }
}

void smith_normal_form(matrix M, matrix U, matrix V) {
 int n = M->n;

 for (int p = 0; p < n; p++) {
  int found = 0;
  int bi = p;
  int bj = p;
  ll best = 0;

  for (int i = p; i < n; i++) {
   for (int j = p; j < n; j++) {
    ll av = ll_abs_snf(M->t[i][j]);
    if (av != 0 && (!found || av < best)) {
```

```
    found = 1;
    best = av;
    bi = i;
    bj = j;
   }
  }
 }

 if (!found)
  break;

 swap_rows(M, U, p, bi);
 swap_cols(M, V, p, bj);

 for (;;) {
  reduce_pivot_cross(M, U, V, p);

  ll piv = M->t[p][p];
  int bad_i = -1;
  for (int i = p + 1; i < n && bad_i < 0; i++) {
   for (int j = p + 1; j < n; j++) {
    if (M->t[i][j] % piv != 0) {
     bad_i = i;
     break;
    }
   }
  }

  if (bad_i < 0)
   break;
  add_row_multiple(M, U, p, bad_i, 1);
 }

 if (M->t[p][p] < 0)
  negate_row(M, U, p);
 }
}

void mat_mul(matrix C, matrix A, matrix B) {
 int n = A->n;
 int i, j, k;

 i = 0;
 while (i < n) {
  j = 0;
  while (j < n) {
   C->t[i][j] = 0;
   j++;
  }
  i++;
 }
 i = 0;
 while (i < n) {
  k = 0;
  while (k < n) {
   ll aik = A->t[i][k];
   if (aik != 0) {
    j = 0;
    while (j < n) {
     C->t[i][j] += aik * B->t[k][j];
     j++;
    }
   }
   k++;
  }
  i++;
 }
```

```
}

matrix get_prod(matrix A, matrix B) {
 matrix C = mat_alloc(A->n);
 mat_mul(C, A, B);
 return C;
}

bool mat_equal(matrix A, matrix B) {
 if (A->n != B->n) return false;
 int n = A->n;
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   if (A->t[i][j] != B->t[i][j]) {
    return false;
   }
  }
 }
 return true;
}

static int det_bareiss_array(const ll *src, int n, ll *det_out) {
 if (n <= 0 || src == NULL || det_out == NULL)
  return -1;
 if (n == 1) {
  *det_out = src[0];
  return 0;
 }

 ll *a = (ll *)malloc((size_t)n * (size_t)n * sizeof(ll));
 if (a == NULL)
  return -1;
 memcpy(a, src, (size_t)n * (size_t)n * sizeof(ll));

 ll prev = 1;
 int sign = 1;

 for (int k = 0; k < n - 1; k++) {
  int pivot = k;
  ll best = ll_abs_snf(a[(size_t)k * (size_t)n + (size_t)k]);
  for (int i = k + 1; i < n; i++) {
   ll av = ll_abs_snf(a[(size_t)i * (size_t)n + (size_t)k]);
   if (av > best) {
    best = av;
    pivot = i;
   }
  }

  if (best == 0) {
   *det_out = 0;
   free(a);
   return 0;
  }

  if (pivot != k) {
   for (int j = 0; j < n; j++) {
    ll tmp = a[(size_t)k * (size_t)n + (size_t)j];
    a[(size_t)k * (size_t)n + (size_t)j] =
     a[(size_t)pivot * (size_t)n + (size_t)j];
    a[(size_t)pivot * (size_t)n + (size_t)j] = tmp;
   }
   sign = -sign;
  }

  ll piv = a[(size_t)k * (size_t)n + (size_t)k];
  for (int i = k + 1; i < n; i++) {
   for (int j = k + 1; j < n; j++) {
```

```c
      ll val = piv * a[(size_t)i * (size_t)n + (size_t)j]
       - a[(size_t)i * (size_t)n + (size_t)k]
        * a[(size_t)k * (size_t)n + (size_t)j];
      a[(size_t)i * (size_t)n + (size_t)j] = val / prev;
     }
    }
    prev = piv;

    for (int i = k + 1; i < n; i++)
     a[(size_t)i * (size_t)n + (size_t)k] = 0;
   }

   *det_out = (ll)sign * a[(size_t)(n - 1) * (size_t)n + (size_t)(n - 1)];
   free(a);
   return 0;
}

static int matrix_det_bareiss(matrix M, ll *det_out) {
   int n = M->n;
   ll *buf = (ll *)malloc((size_t)n * (size_t)n * sizeof(ll));
   if (buf == NULL)
    return -1;
   for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++)
     buf[(size_t)i * (size_t)n + (size_t)j] = M->t[i][j];
   }
   int rc = det_bareiss_array(buf, n, det_out);
   free(buf);
   return rc;
}

static int minor_det_bareiss(matrix M, int skip_row, int skip_col, ll *det_out) {
   int n = M->n;
   if (n == 1) {
    *det_out = 1;
    return 0;
   }

   int m = n - 1;
   ll *minor = (ll *)malloc((size_t)m * (size_t)m * sizeof(ll));
   if (minor == NULL)
    return -1;

   int rr = 0;
   for (int i = 0; i < n; i++) {
    if (i == skip_row)
     continue;
    int cc = 0;
    for (int j = 0; j < n; j++) {
     if (j == skip_col)
      continue;
     minor[(size_t)rr * (size_t)m + (size_t)cc] = M->t[i][j];
     cc++;
    }
    rr++;
   }

   int rc = det_bareiss_array(minor, m, det_out);
   free(minor);
   return rc;
}

int mat_inverse_bareiss(matrix M, matrix adj, ll *det_out) {
   if (M == NULL || adj == NULL || det_out == NULL || M->n <= 0 || adj->n != M->n)
    return -1;

   int n = M->n;
```

```
 if (matrix_det_bareiss(M, det_out) != 0)
  return -1;

 if (*det_out == 0) {
  for (int i = 0; i < n; i++)
   for (int j = 0; j < n; j++)
    adj->t[i][j] = 0;
  return -1;
 }

 if (n == 1) {
  adj->t[0][0] = 1;
  return 0;
 }

 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   ll d = 0;
   if (minor_det_bareiss(M, i, j, &d) != 0)
    return -1;
   if (((i + j) & 1) != 0)
    d = -d;
   adj->t[j][i] = d;
  }
 }
 return 0;
}

matrix mat_inverse(matrix M) {
 if (M == NULL || M->n <= 0)
  return NULL;

 int n = M->n;
 matrix W = mat_alloc(n);
 matrix Inv = mat_alloc(n);
 if (W == NULL || Inv == NULL) {
  mat_free(W);
  mat_free(Inv);
  return NULL;
 }

 for (int i = 0; i < n; i++)
  memcpy(W->t[i], M->t[i], (size_t)n * sizeof(ll));
 mat_identity(Inv);

 for (int k = 0; k < n; k++) {
  int pivot_row = -1;
  for (int i = k; i < n; i++)
  {
   if (W->t[i][k] != 0) {
    pivot_row = i;
    break;
   }
  }
  if (pivot_row == -1) {
   mat_free(W);
   mat_free(Inv);
   return NULL;
  }
  swap_rows(W, Inv, k, pivot_row);

  for (int i = k + 1; i < n; i++) {
   while (W->t[i][k] != 0) {
    ll pivot = W->t[k][k];
    ll val = W->t[i][k];
    ll x, y, g;
```

```
    ll sp = (pivot < 0) ? -1 : 1;
    ll sv = (val < 0) ? -1 : 1;
    ll ap = pivot * sp;
    ll av = val * sv;
    g = extended_gcd(ap, av, &x, &y);
    x *= sp;
    y *= sv;

    row_combine(W, Inv, k, i, x, y, -(val / g), pivot / g);
   }
  }

  if (W->t[k][k] != 1 && W->t[k][k] != -1) {
   mat_free(W);
   mat_free(Inv);
   return NULL;
  }

  if (W->t[k][k] == -1) {
   for (int j = 0; j < n; j++) {
    W->t[k][j] = -W->t[k][j];
    Inv->t[k][j] = -Inv->t[k][j];
   }
  }

  for (int i = 0; i < n; i++) {
   if (i == k || W->t[i][k] == 0)
    continue;
   ll factor = W->t[i][k];
   for (int j = 0; j < n; j++) {
    W->t[i][j] -= factor * W->t[k][j];
    Inv->t[i][j] -= factor * Inv->t[k][j];
   }
  }
 }

 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   ll expected = (i == j) ? 1 : 0;
   if (W->t[i][j] != expected) {
    mat_free(W);
    mat_free(Inv);
    return NULL;
   }
  }
 }

 mat_free(W);
 return Inv;
}

matrix mat_snf_V_inv(matrix R) {
    int n = R->n;
    matrix U = mat_alloc(n);
    matrix V = mat_alloc(n);
    matrix M = mat_alloc(n);

    int i = 0;
    while (i < n) {
        memcpy(M->t[i], R->t[i], (size_t)n * sizeof(ll));
        i++;
    }

    mat_identity(U);
    mat_identity(V);
    smith_normal_form(M, U, V);
```

```
    matrix V_inv = mat_inverse(V);

    mat_free(U);
    mat_free(V);
    mat_free(M);
    return V_inv;
}

void mat_snf_with_diagonal(matrix M, matrix U, matrix V, matrix D) {
    int n = M->n;
    int i = 0;

    while (i < n) {
        memcpy(D->t[i], M->t[i], (size_t)n * sizeof(ll));
        i++;
    }

    mat_identity(U);
    mat_identity(V);
    smith_normal_form(D, U, V);
}

bool same_abelian_group_from_snf(matrix D1, matrix D2) {
    int i = 0;
    int j = 0;

    while (i < D1->n || j < D2->n) {
        while (i < D1->n && abs(D1->t[i][i]) == 1) i++;
        while (j < D2->n && abs(D2->t[j][j]) == 1) j++;

        if (i >= D1->n || j >= D2->n) {
            return i >= D1->n && j >= D2->n;
        }

        if (abs(D1->t[i][i]) != abs(D2->t[j][j])) {
            return false;
        }

        i++;
        j++;
    }

    return true;
}
```

```
#ifndef GEN_TBL
#define GEN_TBL

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <assert.h>
#include "../Linked_List/linked_list.h"

typedef struct data_s {
 int g;
 int p;
} data_t;
typedef data_t* data;

typedef struct node_data_s {
 int g;
 int p;
 struct node_data_s* next;
} node_data_t;

typedef node_data_t* node_data;

typedef struct data_list_s {
  node_data head;
  node_data tail;
  int length;
} data_list_t;
typedef data_list_t* data_list;

node_data new_data_node(int g, int p);
bool is_empty_data_list(data_list dlist);

data_list create_empty_data_list();
data_list create_example_data_list(int n);
data_list create_single_data_list(int n);
data_list deep_copy_data_list(data_list dlist);
void del_copy_from(data_list* d1, data_list d2);

void show_values_data_list(data_list dlist);
void push_data_list(data_list dlist, int g, int p);
void append_data_list(data_list dlist, int g, int p);
data pop_data_list(data_list dlist);

node_data free_get_next_node_data(node_data node);
void free_data_list(data_list dlist);

typedef struct group_data_s {
 int n;
 bool initialised;

 bool* contains;
 list grp_elts;

 int* order;
 data_list generators;

 data_list* decomps;
} group_data_t;
typedef group_data_t* group_data;

group_data create_empty_group_data(int n);
void init_empty_group_data(group_data gd, int neutral);
void free_group_data(group_data tr);

void show_num_elts(group_data tr);
void show_rel(group_data tr);
```

```c
void show_contains(group_data tr);
void show_grp_elts(group_data tr);
void show_generators(group_data tr);

#endif
```

```c
#include "gen_table.h"

data create_data(int g, int p) {
 data d = malloc(sizeof(data_t));
 d->g = g;
 d->p = p;
 return d;
}

node_data new_data_node(int g, int p) {
 node_data node = malloc(sizeof(node_data_t));
 node->g = g;
 node->p = p;
 node->next = NULL;
 return node;
}

data_list create_empty_data_list() {
 data_list new_data_list = malloc(sizeof(data_list_t));
 new_data_list->head = NULL;
 new_data_list->tail = NULL;
 new_data_list->length = 0;
 return new_data_list;
}

data_list create_example_data_list(int n) {
 data_list example = create_empty_data_list();
 for (int i = 0; i < n; i++) {
  push_data_list(example, i, i);
 }
 return example;
}

data_list create_single_data_list(int g) {
 data_list single = create_empty_data_list();
 push_data_list(single, g, 1);
 return single;
}

data_list deep_copy_data_list(data_list dlist) {
 data_list copy = create_empty_data_list();
 for (node_data c=dlist->head; c != NULL; c=c->next) {
  append_data_list(copy, c->g, c->p);
 }

 return copy;
}

void del_copy_from(data_list* d1, data_list d2) {
 assert(d1 != NULL && d2 != NULL);
 free_data_list(*d1);
 *d1 = deep_copy_data_list(d2);
}

bool is_empty_data_list(data_list dlist) {
 assert(dlist != NULL);
 return (dlist->length == 0 && dlist->head == NULL);
}

void show_values_data_list(data_list dlist) {
 assert(dlist != NULL);
 for (node_data c = dlist->head; c != NULL; c=c->next) {
  printf("(%d,%d) -> ", c->g, c->p);
 }
 printf("END\n");
}
```

```c
void push_data_list(data_list dlist, int g, int p) {
 // printf("Adding: (%d, %d)\n", g, p);

 node_data new_head = new_data_node(g, p);
 node_data old_head = dlist->head;
 new_head->next = old_head;
 dlist->head = new_head;
 dlist->length += 1;

 if (dlist->length == 1) {
  dlist->tail = dlist->head;
 }
}

void append_data_list(data_list dlist, int g, int p) {
 if (dlist->length <= 0) {
  push_data_list(dlist, g, p);
 } else {
  node_data new_tail = new_data_node(g, p);
  dlist->tail->next = new_tail;
  dlist->tail = new_tail;
  dlist->length += 1;
 }
}

data pop_data_list(data_list dlist) {
 assert(dlist != NULL && !is_empty_data_list(dlist));
 node_data old_head = dlist->head;
 node_data new_head = old_head->next;
 dlist->head = new_head;

 data res = create_data(old_head->g, old_head->p);
 free(old_head);
 dlist->length--;
 return res;
}

node_data free_get_next_node_data(node_data node) {
 assert(node != NULL);
 node_data next = node->next;
 free(node);
 return next;
}

void free_data_list(data_list dlist) {
 assert(dlist != NULL);
 node_data curr = dlist->head;
 while (curr != NULL) {
  curr = free_get_next_node_data(curr);
 }

 free(dlist);
}

group_data create_empty_group_data(int n) {
 group_data res = malloc(sizeof(group_data_t));
 res->n = n;
 res->initialised = false;


 res->contains = calloc(n, sizeof(bool));
 res->order = malloc(n*sizeof(int));
 for (int i = 0; i < n; i++) {
  res->order[i] = -1;
 }

 res->grp_elts = create_empty_list();
```

```
 res->generators = create_empty_data_list();

 res->decomps = malloc(n*sizeof(data_list));
 for (int i = 0; i < n; i++) {
  res->decomps[i] = create_empty_data_list();
 }

 return res;
}

void init_empty_group_data(group_data gd, int neutral) {
 assert(gd != NULL && neutral >= 0 && neutral < gd->n);
 assert(!gd->initialised);
 assert(gd->generators->length == 0 && gd->grp_elts->length == 0);
 push(gd->grp_elts, neutral);
 gd->contains[neutral] = true;
 gd->initialised = true;
}

void free_group_data(group_data tr) {
 assert(tr != NULL);
 for (int i = 0; i < tr->n; i++) {
  free_data_list(tr->decomps[i]);
 }
 free(tr->decomps);
 free_list(tr->grp_elts);
 free_data_list(tr->generators);
 free(tr->order);
 free(tr->contains);
 free(tr);
}

void show_num_elts(group_data tr) {
 int num_elts = 0;
 for (int i = 0; i < tr->n; i++) {
  if (tr->contains[i]) {
   num_elts++;
  }
 }
 printf("Nombre d'éléménts: %d\n", num_elts);
}

void show_rel(group_data tr) {
 printf("Décompositions:\n");
 assert(tr != NULL && tr->decomps != NULL);
 for (int i = 0; i < tr->n; i++) {
  if (tr->decomps[i]->length >= 1) {
   printf("%d: ", i);
   show_values_data_list(tr->decomps[i]);
  }
 }
}

void show_contains(group_data tr) {
 assert(tr != NULL && tr->contains != NULL);
 for (int i = 0; i < tr->n; i++) {
  printf("Has %d: ", i);
  if (tr->contains[i]) {
   printf("TRUE\n");
  } else {
   printf("FALSE\n");
  }
 }
}

void show_grp_elts(group_data tr) {
 assert(tr != NULL && tr->grp_elts != NULL);
```

```
 show_values(tr->grp_elts);
}

void show_generators(group_data tr) {
 assert(tr != NULL && tr->generators != NULL);
 printf("Générateur(s): ");
 show_values_data_list(tr->generators);
}
```

```c
#ifndef GARBAGE_COLLECTOR_H
#define GARBAGE_COLLECTOR_H

#include "../Gen_Table/gen_table.h"

typedef struct gbg_node_s {
 node_data* to_free;
 struct gbg_node_s* next;
} gbg_node_t;
typedef gbg_node_t* gbg_node;

typedef struct gbg_collector_s {
 gbg_node head;
 int size;
} gbg_collector_t;
typedef gbg_collector_t* gbg_collector;

gbg_node create_gbg_node(node_data* nd_addr);
void free_gbg_node(gbg_node head_gbg_node);

gbg_collector create_empty_gbg_collector(void);
void push_gbg_collector(node_data* nd_addr, gbg_collector collector);
void free_gbg_collector(gbg_collector collector);
void print_gbg_collector(gbg_collector collector);

void free_decomp_table(data_list* tabl, int n, gbg_collector coll);

// New functions to append in decomp table.
void append_data_list_from_ptr(data_list dlist, node_data* ptr);
void append_data_in_tabl_only(data_list* tabl, int n, int i, int g, int p, gbg_collector
coll);
void append_data_and_link_in_tabl(data_list* tabl, int n, int i, int j, int g, int p,
gbg_collector coll);

void free_gbc_gdata(group_data gd, gbg_collector gbc);


#endif
```

```c
#include "garbage_collector.h"

gbg_node create_gbg_node(node_data* nd_addr) {
 gbg_node res = malloc(sizeof(gbg_node_t));
 res->to_free = nd_addr;
 res->next = NULL;

 return res;
}

void free_gbg_node(gbg_node head_gbg_node) {
 if (head_gbg_node == NULL) return;
 gbg_node next = head_gbg_node->next;
 free(*(head_gbg_node->to_free));
 free(head_gbg_node->to_free);
 free(head_gbg_node);

 free_gbg_node(next);
}

gbg_collector create_empty_gbg_collector(void) {
 gbg_collector res = malloc(sizeof(gbg_collector_t));
 res->head = NULL;
 res->size = 0;
 return res;
}

void push_gbg_collector(node_data* nd_addr, gbg_collector collector) {
 gbg_node new_head = create_gbg_node(nd_addr);
 new_head->next = collector->head;
 collector->head = new_head;
 collector->size++;
}

void free_gbg_collector(gbg_collector collector) {
 free_gbg_node(collector->head);
 free(collector);
}

void print_gbg_collector(gbg_collector collector) {
 assert(collector != NULL);
 printf("To free: %d\n", collector->size);
}

void free_decomp_table(data_list* tabl, int n, gbg_collector coll) {
 free_gbg_collector(coll);
 for (int i = 0; i < n; i++) {
  free(tabl[i]);
 }
 free(tabl);
}

void append_data_list_from_ptr(data_list dlist, node_data* ptr) {
 assert(dlist != NULL);
 if (dlist->length == 0) {
  dlist->head = *ptr;
  dlist->tail = *ptr;
 } else {
  dlist->tail->next = *ptr;
  dlist->tail = *ptr;
 }
 dlist->length++;
}

void append_data_in_tabl_only(data_list* tabl, int n, int i, int g, int p, gbg_collector coll)
{
 assert(tabl != NULL && i >= 0 && i < n);
```

```c
 // Allocate fresh memory in the heap.
 node_data* ptr = malloc(sizeof(node_data));
 *ptr = new_data_node(g, p);

 append_data_list_from_ptr(tabl[i], ptr);

 push_gbg_collector(ptr, coll);

 // printf("Added value at index %d: (%d, %d)\n", i, g, p);
 // printf("Address of new tail in tabl: %p\n", ptr);
}

void append_data_and_link_in_tabl(data_list* tabl, int n, int i, int j, int g, int p,
gbg_collector coll) {
 assert(tabl != NULL && i != j);
 assert(i >= 0 && i < n && j >= 0 && j < n);

 // Allocate fresh memory in the heap.
 node_data* ptr = malloc(sizeof(node_data));
 *ptr = new_data_node(g, p);

 append_data_list_from_ptr(tabl[i], ptr);

 tabl[i]->tail->next = tabl[j]->head;

 push_gbg_collector(ptr, coll);


 // printf("Added value at index %d: (%d, %d)\n", i, g, p);
 // printf("Address of new tail in tabl: %p\n", ptr);
}

void free_gbc_gdata(group_data gd, gbg_collector gbc) {
 // free_gbg_collector(gbc);
 // for (int i = 0; i < gd->n; i++) {
 //  free(gd->decomps[i]);
 // }
 // free(gd->decomps);
 free_decomp_table(gd->decomps, gd->n, gbc);
 free_list(gd->grp_elts);
 free_data_list(gd->generators);
 free(gd->order);
 free(gd->contains);
 free(gd);
}
```

```c
#ifndef LL
#define LL

#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <stdbool.h>

typedef struct node {
  int val;
  struct node* next;
} node_t;

typedef struct list {
  node_t* head;
  node_t* tail;
  int length;
} list_t;
typedef list_t* list;

node_t* new_node(int n);

list_t* create_empty_list();
list_t* create_example(int n);
list_t* create_single(int n);

list_t* deep_copy(list_t* l);

bool is_empty(list_t* linked);

void show_list(list_t* linked);
void show_values(list_t* linked);

void push(list_t* linked, int n);
void append(list_t* linked, int n);
void insert(list_t* linked, int index, int x);
int pop(list_t* linked);

node_t* free_get_next(node_t* node);
void free_list(list_t* linked);

#endif
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <stdbool.h>

#include "linked_list.h"

node_t* new_node(int n) {
    node_t* out = malloc(sizeof(node_t));
    out->next = NULL;
    out->val = n;
    return out;
}

list_t* create_empty_list() {
    list_t* linked = malloc(sizeof(list_t));
    linked->head = NULL;
    linked->tail = NULL;
    linked->length = 0;
    return linked;
}

list_t* create_example(int n) {
    list_t* linked = create_empty_list();
    for (int i = 0; i < n; i++) {
        append(linked, i);
    }
    return linked;
}

list_t* create_single(int n) {
    list_t* linked = create_empty_list();
    push(linked, n);
    return linked;
}

list_t* deep_copy(list_t* l) {
    list_t* copy = create_empty_list();
    node_t* to_append;
    for (node_t* c = l->head; c != NULL; c = c->next) {
        append(copy, c->val);
    }

    return copy;
}

bool is_empty(list_t* linked) {
    return (linked->length == 0);
}

void show_list(list_t* linked) {
    printf("TĂŞte: %d\n", linked->head->val);
    printf("Queue: %d\n", linked->tail->val);
    printf("Longueur: %d\n", linked->length);

    node_t* curr = linked->head;
    while (curr != NULL) {
        printf("%d -> ", curr->val);
        curr = curr->next;
    }
    printf("FIN\n");
}

void show_values(list_t* linked) {
    for (node_t* c = linked->head; c != NULL; c = c->next) {
        printf("%d -> ", c->val);
    }
```

```c
    printf("NULL\n");
}

void push(list_t* linked, int n) {
    node_t* new_head = malloc(sizeof(node_t));
    new_head->val = n;

    node_t* old_head = linked->head;
    new_head->next = old_head;
    linked->head = new_head;
    linked->length += 1;

    if (linked->length == 1) {
        linked->tail = new_head;
    }
}

void append(list_t* linked, int n) {
    assert(linked != NULL);
    if (linked->length == 0) {
        node_t* new_tail = new_node(n);
        linked->tail = new_tail;
        linked->head = new_tail;
        linked->length++;
    } else {
        node_t* new_tail = malloc(sizeof(node_t));
        new_tail->val = n;
        new_tail->next = NULL;

        node_t* old_tail = linked->tail;
        old_tail->next = new_tail;
        linked->tail = new_tail;
        linked->length += 1;
    }
}

void insert(list_t* linked, int index, int x) {
    assert(index <= linked->length);
    if (index == 0) {
        push(linked, x);
    }
    if (index == linked->length) {
        append(linked, x);
    }
    else {
        node_t* new_node = malloc(sizeof(node_t));
        new_node->val = x;

        node_t* curr = linked->head;
        int i = 0;
        while (i < index-1) {
            curr = curr->next;
            i += 1;
        }
        new_node->next = curr->next;
        curr->next = new_node;

        linked->length += 1;
    }
}

int pop(list_t* linked) {
    assert(is_empty(linked) == false);
    node_t* new_head = linked->head->next;
    node_t* old_head = linked->head;
    int val = old_head->val;
    linked->head = new_head;
```

```
    free(old_head);
    linked->length--;
    return val;
}

node_t* free_get_next(node_t* node) {
    assert(node != NULL);
    node_t* next = node->next;
    free(node);
    return next;
}

void free_list(list_t* linked) {
    assert(linked != NULL);
    node_t* curr = linked->head;
    while (curr != NULL) {
        curr = free_get_next(curr);
    }
    free(linked);
}
```

```c
#include "group.h"

int *zero_array(int n) {
 int *res = malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) {
  res[i] = 0;
 }
 return res;
}

void group_name(group_t *g, char *s) {
 g->name = s;
}

group group_empty(int n) {
 group new_empty_group = malloc(sizeof(group_t));
 new_empty_group->n = n;
 new_empty_group->table = malloc(n * sizeof(int *));
 for (int i = 0; i < n; i++) {
  new_empty_group->table[i] = zero_array(n);
 }
 return new_empty_group;
}

void group_free(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 for (int i = 0; i < n; i++) {
  free(g->table[i]);
 }
 free(g->table);
 free(g);
}


void group_print(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
  int n = g->n;
 printf("Groupe %s d'ordre %d\n", g->name, n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   printf("%d ", g->table[i][j]);
  }
  printf("\n");
 }
}

group from_matrix(int **m) {
 assert(m != NULL && "[-] NULL POINTER exception.\n");
 int n = sizeof(*m[0]) / sizeof(int);
 group res = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   res->table[i][j] = m[i][j];
  }
 }
 return res;
}

group copy_group(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 group copy = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   copy->table[i][j] = g->table[i][j];
  }
```

```
 }
 return copy;
}

int calculate(group g, int x, int y) {
 assert(y >= 0);
 if (x == -1) {
  return y;
 }
 return g->table[x][y];
}

bool is_neutral(group g, int x) {
 assert(g != NULL);
 assert(x >= 0 && x < g->n);
 int res;
 if (x < g->n - 1) {
  res = calculate(g, x, x+1);
  return (res == x+1);
 } else {
  res = calculate(g, x, 0);
  return (res == 0);
 }
}

int get_neutral(group  g) {
 assert(g != NULL);
 for (int i = 0; i < g->n; i++) {
  if (is_neutral(g, i)) {
   return i;
  }
 }
 printf("Erreur.\n");
 return -1;
}
```

```c
#include "group.h"

int *zero_array(int n) {
 int *res = malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) {
  res[i] = 0;
 }
 return res;
}

void group_name(group_t *g, char *s) {
 g->name = s;
}

group group_empty(int n) {
 group new_empty_group = malloc(sizeof(group_t));
 new_empty_group->n = n;
 new_empty_group->table = malloc(n * sizeof(int *));
 for (int i = 0; i < n; i++) {
  new_empty_group->table[i] = zero_array(n);
 }
 return new_empty_group;
}

void group_free(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 for (int i = 0; i < n; i++) {
  free(g->table[i]);
 }
 free(g->table);
 free(g);
}


void group_print(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
  int n = g->n;
 printf("Groupe %s d'ordre %d\n", g->name, n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   printf("%d ", g->table[i][j]);
  }
  printf("\n");
 }
}

group from_matrix(int **m) {
 assert(m != NULL && "[-] NULL POINTER exception.\n");
 int n = sizeof(*m[0]) / sizeof(int);
 group res = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   res->table[i][j] = m[i][j];
  }
 }
 return res;
}

group copy_group(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 group copy = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   copy->table[i][j] = g->table[i][j];
  }
```

```c
 }
 return copy;
}

int calculate(group g, int x, int y) {
 assert(y >= 0);
 if (x == -1) {
  return y;
 }
 return g->table[x][y];
}

bool is_neutral(group g, int x) {
 assert(g != NULL);
 assert(x >= 0 && x < g->n);
 int res;
 if (x < g->n - 1) {
  res = calculate(g, x, x+1);
  return (res == x+1);
 } else {
  res = calculate(g, x, 0);
  return (res == 0);
 }
}

int get_neutral(group  g) {
 assert(g != NULL);
 for (int i = 0; i < g->n; i++) {
  if (is_neutral(g, i)) {
   return i;
  }
 }
 printf("Erreur.\n");
 return -1;
}
```